Plataformas de Desenvolvimento CI/CD

24/25

Ricardo Filipe (ricardo.filipe@isec.pt)

Mestrado em Informática e Sistemas

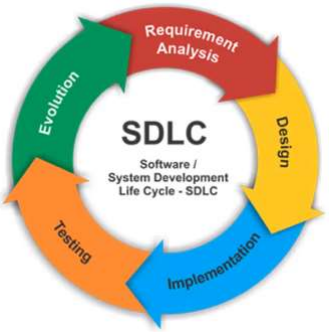
1

General Objectives

- Gain knowledge about Continous Integration / Continous Delivery (CI/CD)
- Hands-on – create a pipeline CI/CD

2

Old Process



Very Slow
Couldn't handle rapid changes in business

3

Delivery Flow



SW team

4

Delivery Flow



SW team



5

Delivery Flow



SW team





Build and integration team

6

Delivery Flow



SW team





Build and integration team



Compile

7

Delivery Flow



SW team





Build and integration team



Compile



Package

8

Deployment Flow



Package

9

Deployment Flow



Package



Instructions

10

Deployment Flow



Package



Instructions





Operations Team

11

Deployment Flow



Package



Instructions





Operations Team

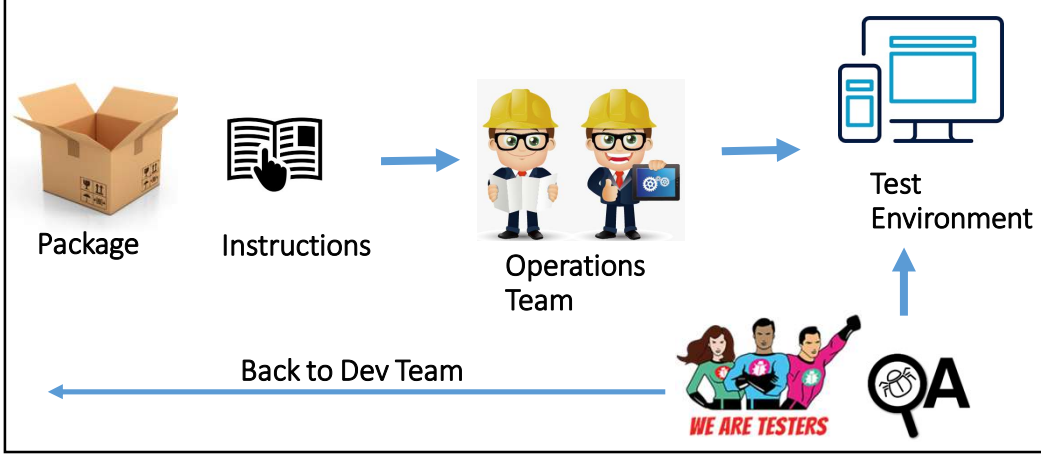


Test Environment



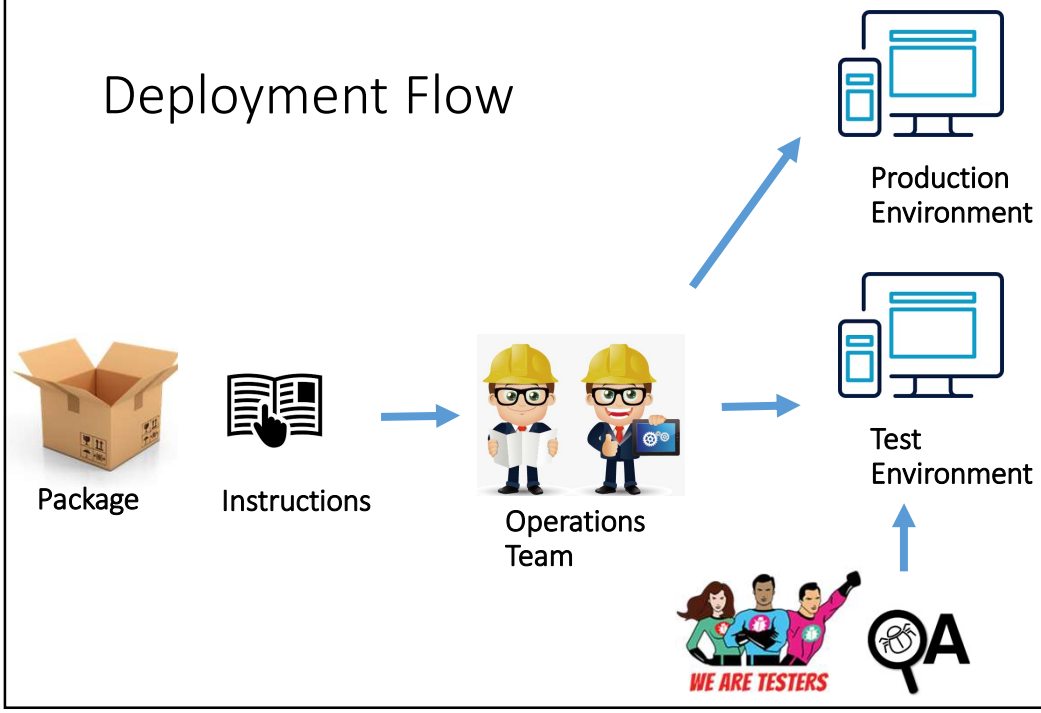
12

Deployment Flow



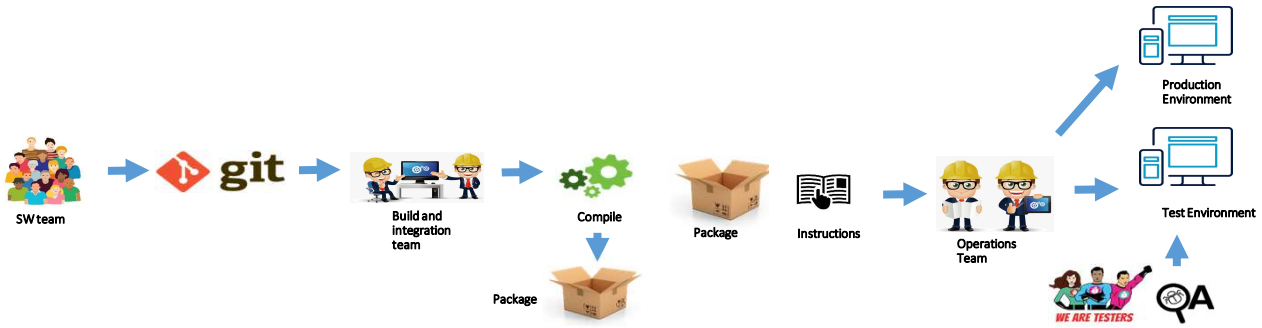
13

Deployment Flow



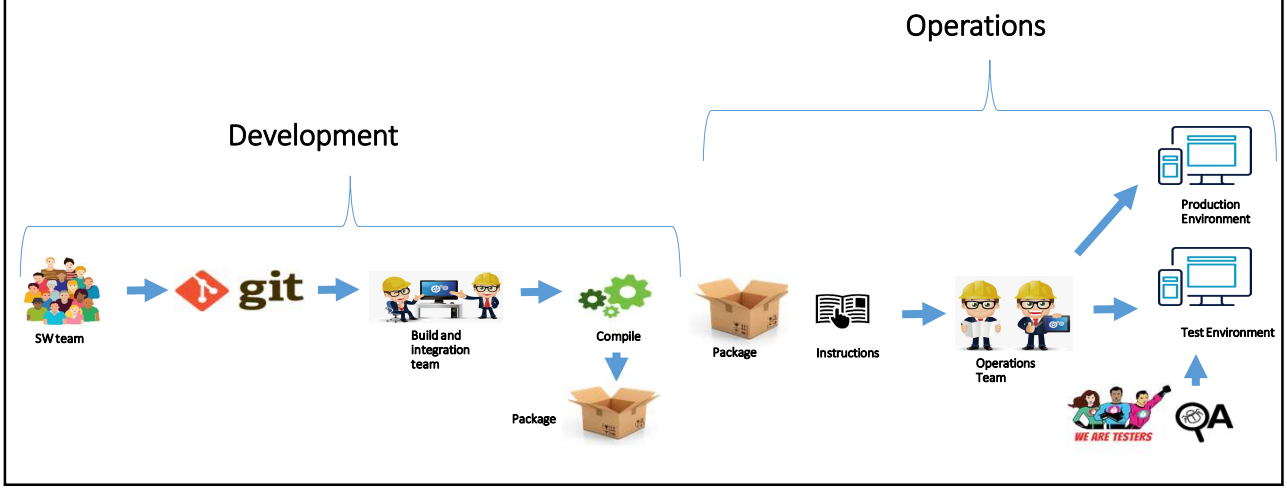
14

Deployment Flow



15

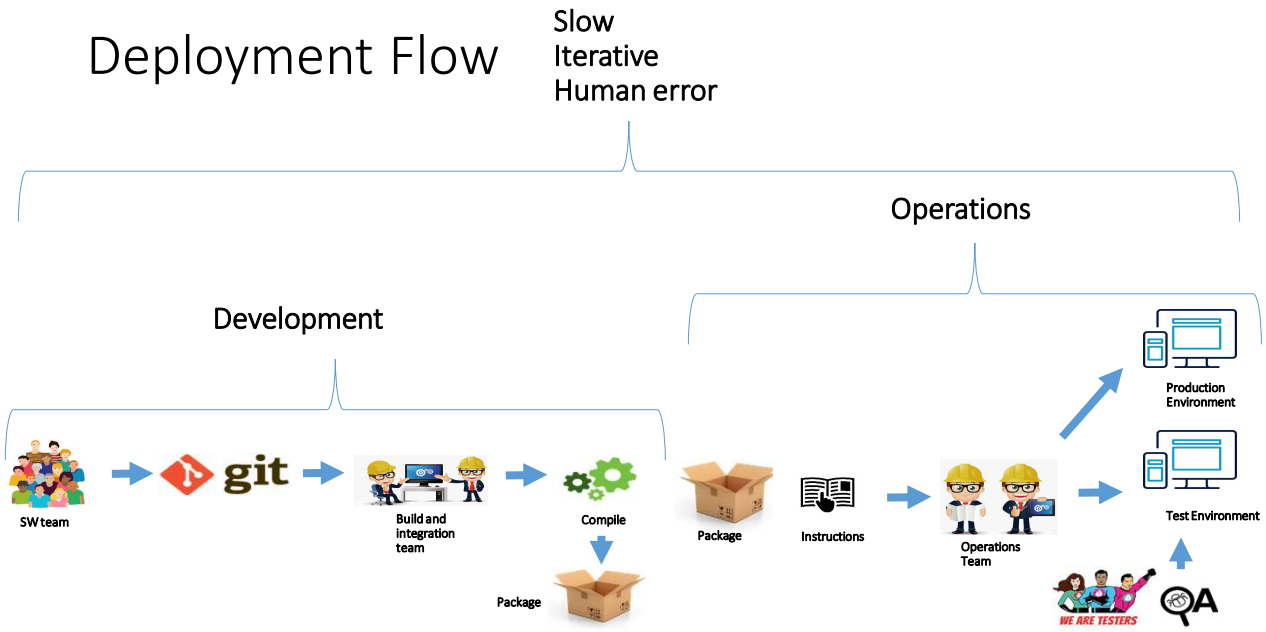
Deployment Flow



16

Deployment Flow

Slow
Iterative
Human error



17

Main issues

- Slow iterations (weeks to months)
- Integration is difficult specially with multi component system
- Fix packaging at the end of the development cycle
- Merges only at the end of development cycle, blocking development team
- Bug feedback slow to development team

18

Main issues

- Instructions must be correct to ensure the SW is well installed.
- Instructions might be diferent according to the environment.
- Human prone – e.g. manual tasks.
- Small number of deployments per year. Difficult, high downtime, huge impact.

19

Continuous Integration

Continuous Integration (**CI**) is the practice of merging all developers' working copies to a shared mainline several times a day.

20

Continuous Integration



SW team





Build and integration team



Compile



Package

21

Continuous Integration



SW team





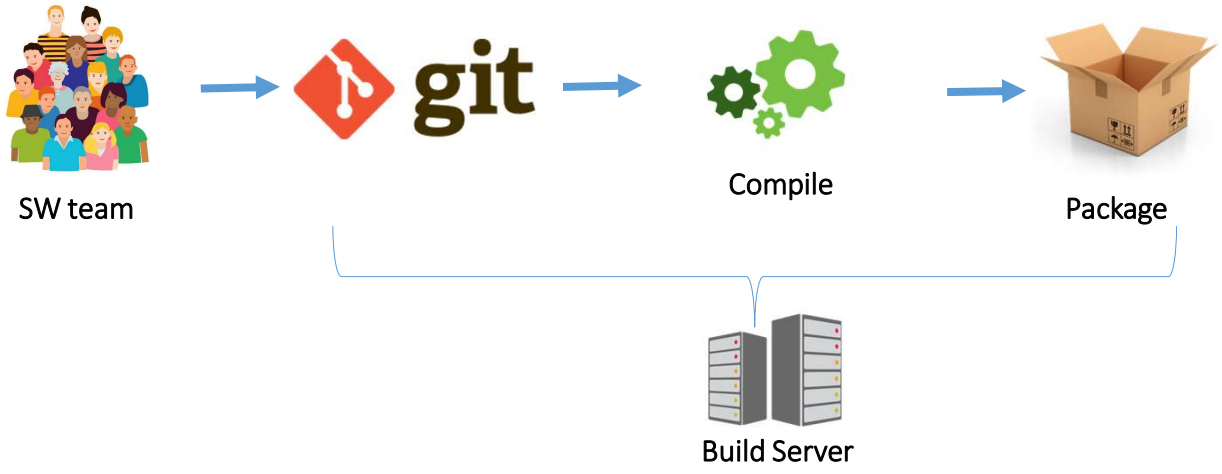
Compile



Package

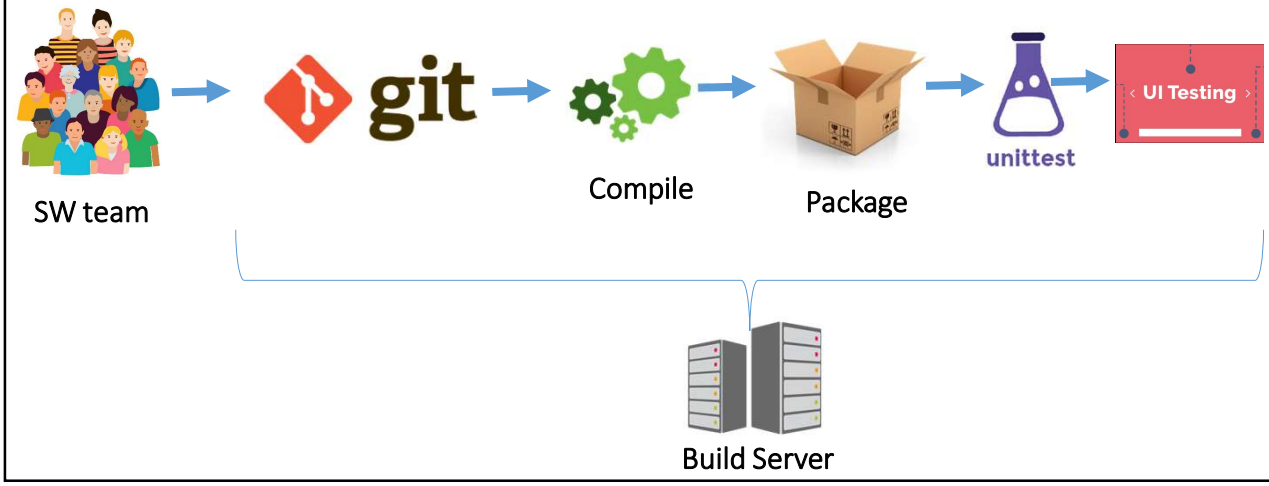
22

Continuous Integration



23

Continuous Integration



24

Continuous Integration

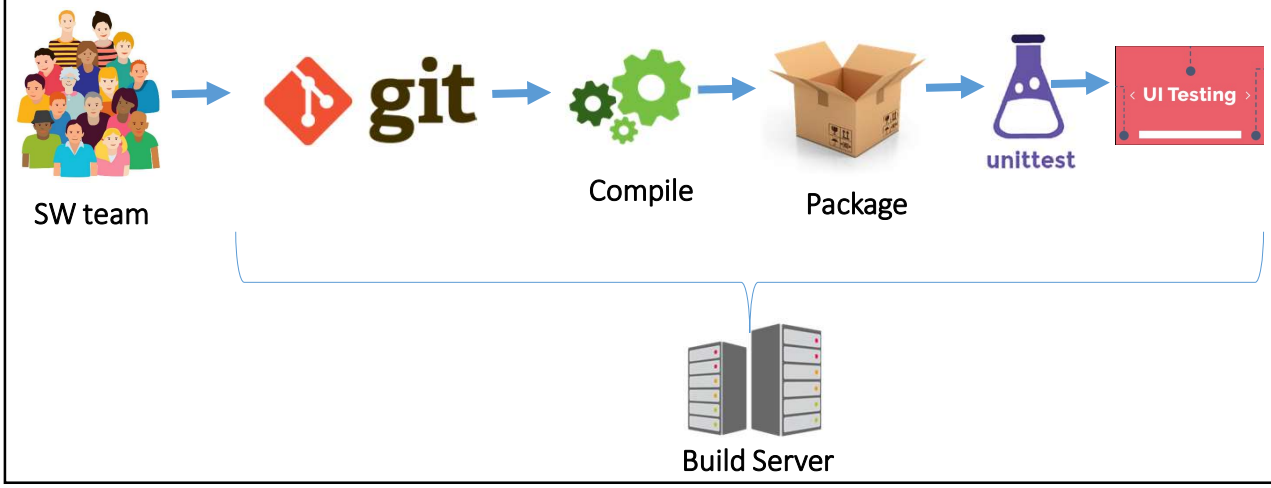
Single repositor

Frequent commits by the developers

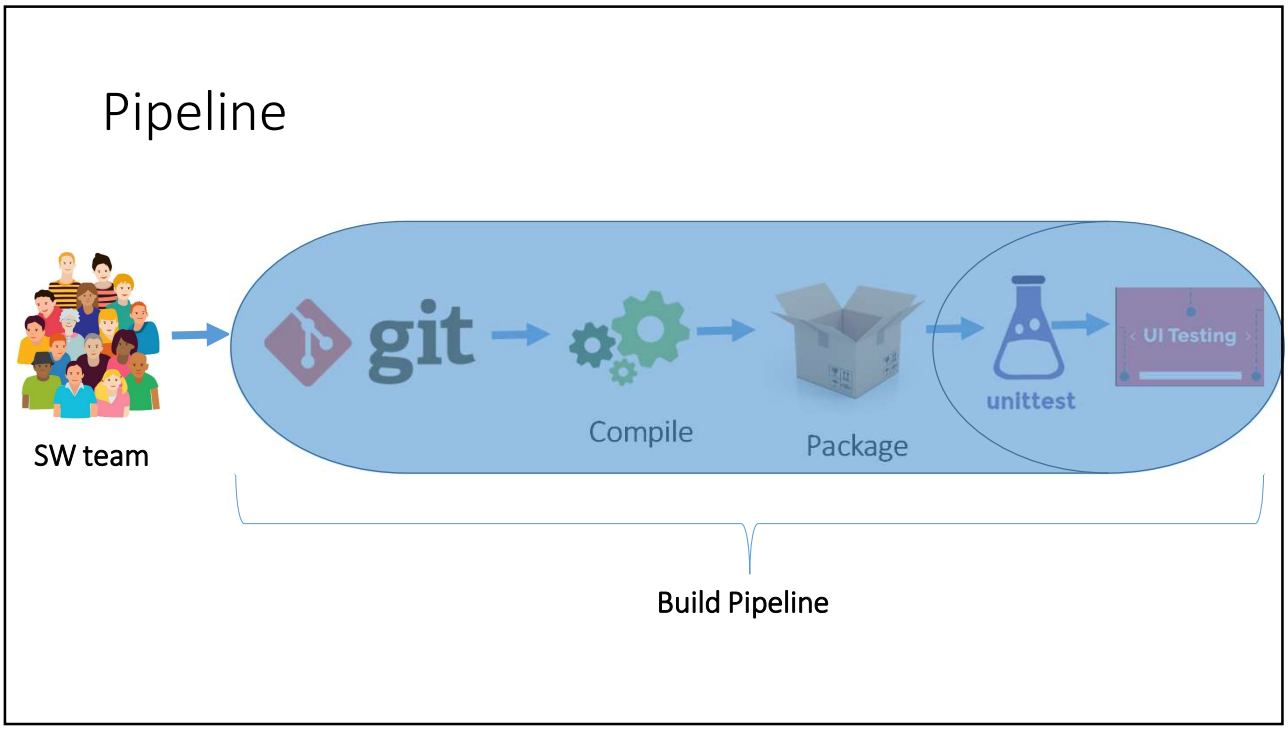
Build Server responsible for:
- Compile code automatically
- Dashboards to understand code quality
- Notify Developers for bugs
- Run unity, system and UI automated tests
- Bug fix (backlog) with higher priority than new features

25

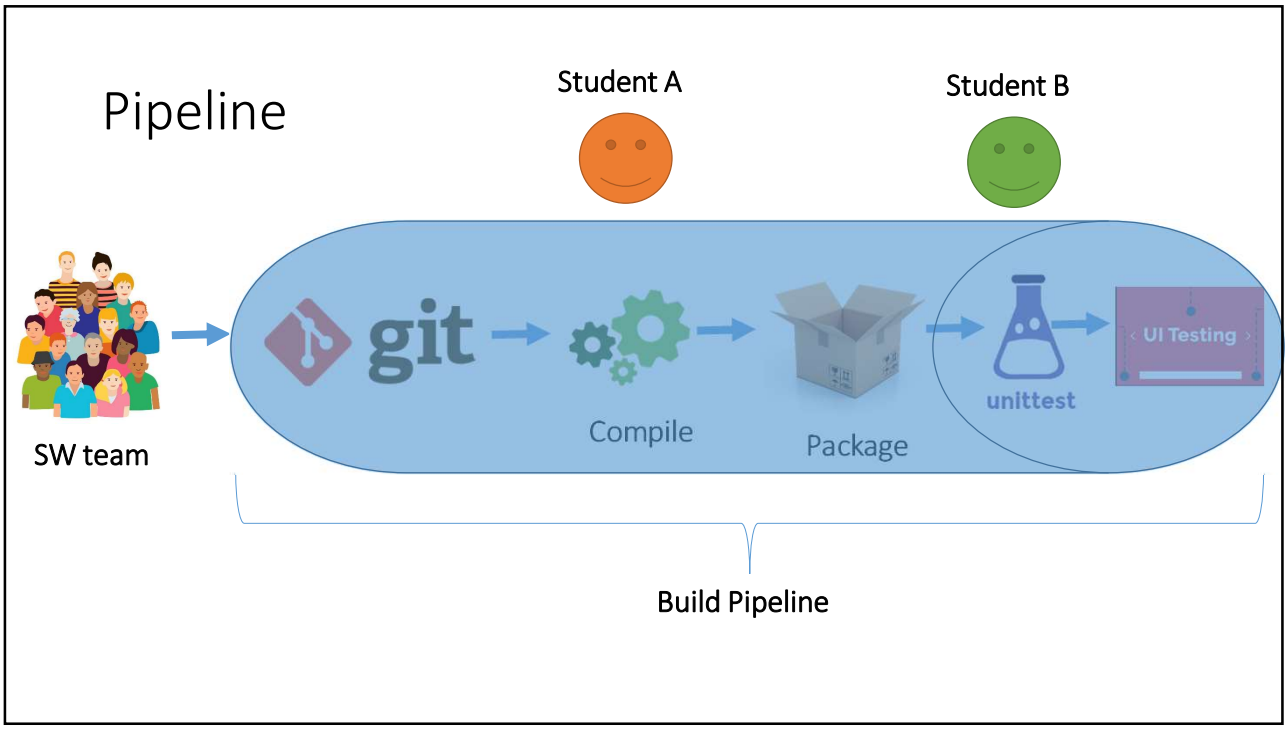
Pipeline



26

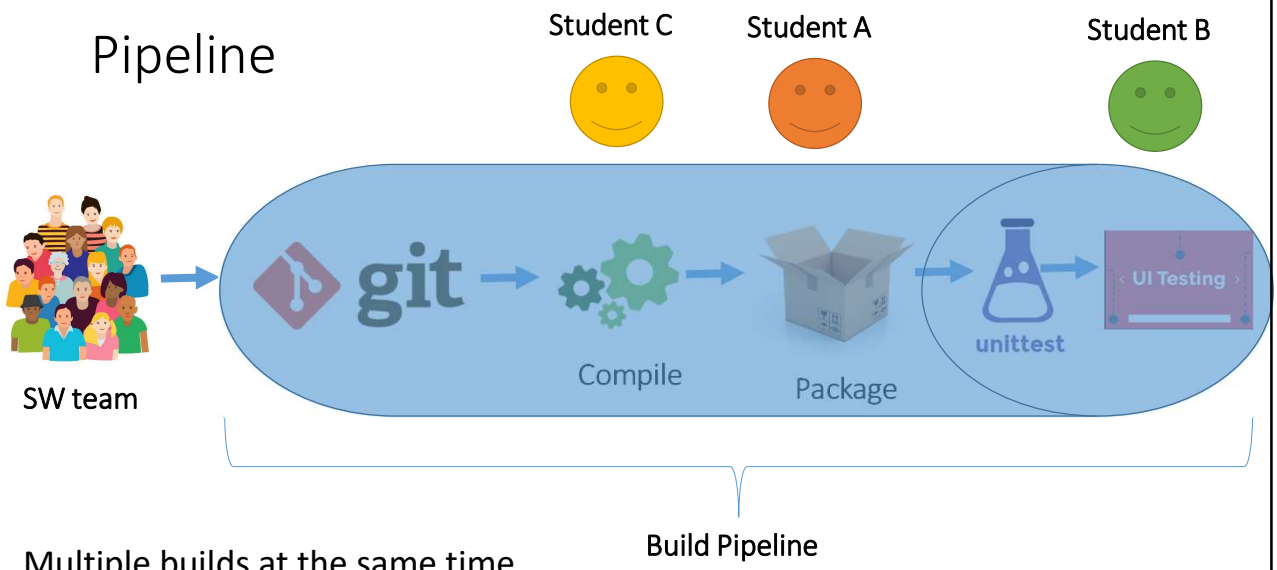

Pipeline
SW team
git
Compile
Package
unittest
UI Testing
Build Pipeline

27


Pipeline
Student A
Student B
SW team
git
Compile
Package
unittest
UI Testing
Build Pipeline

28

Pipeline



Multiple builds at the same time.

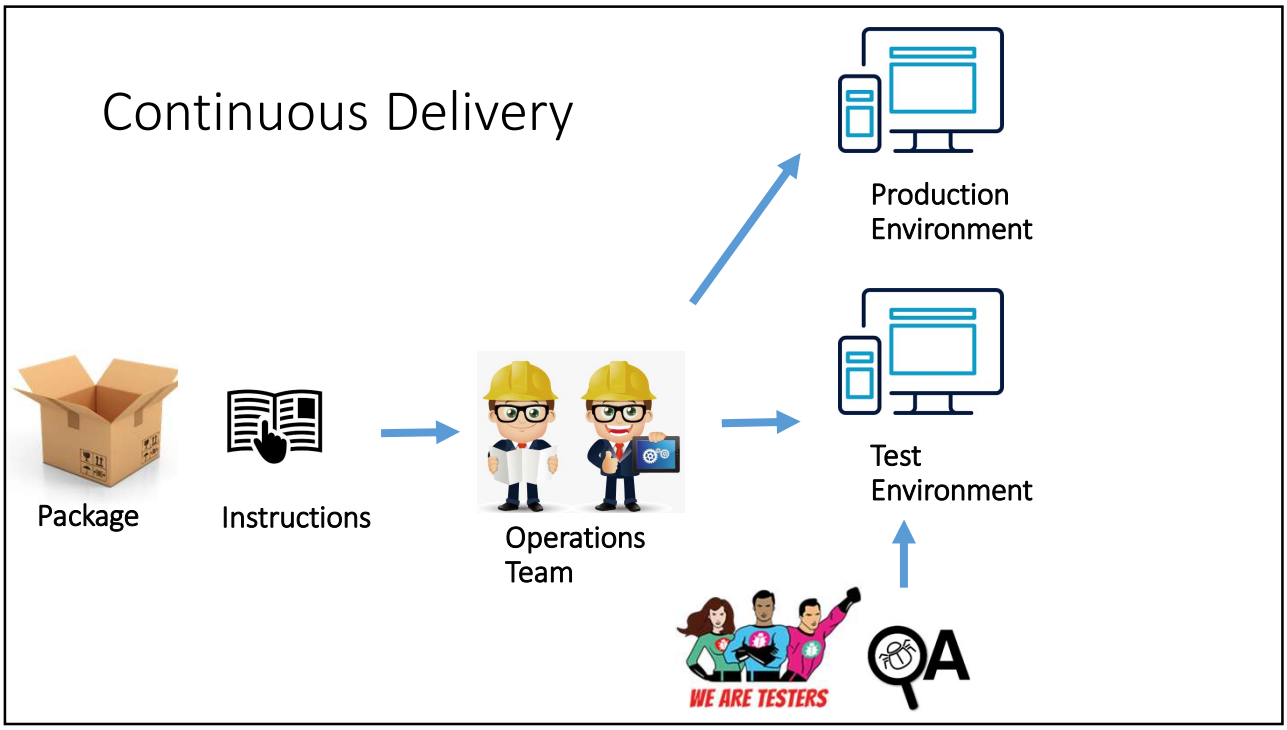
Assemble line methodology.

29

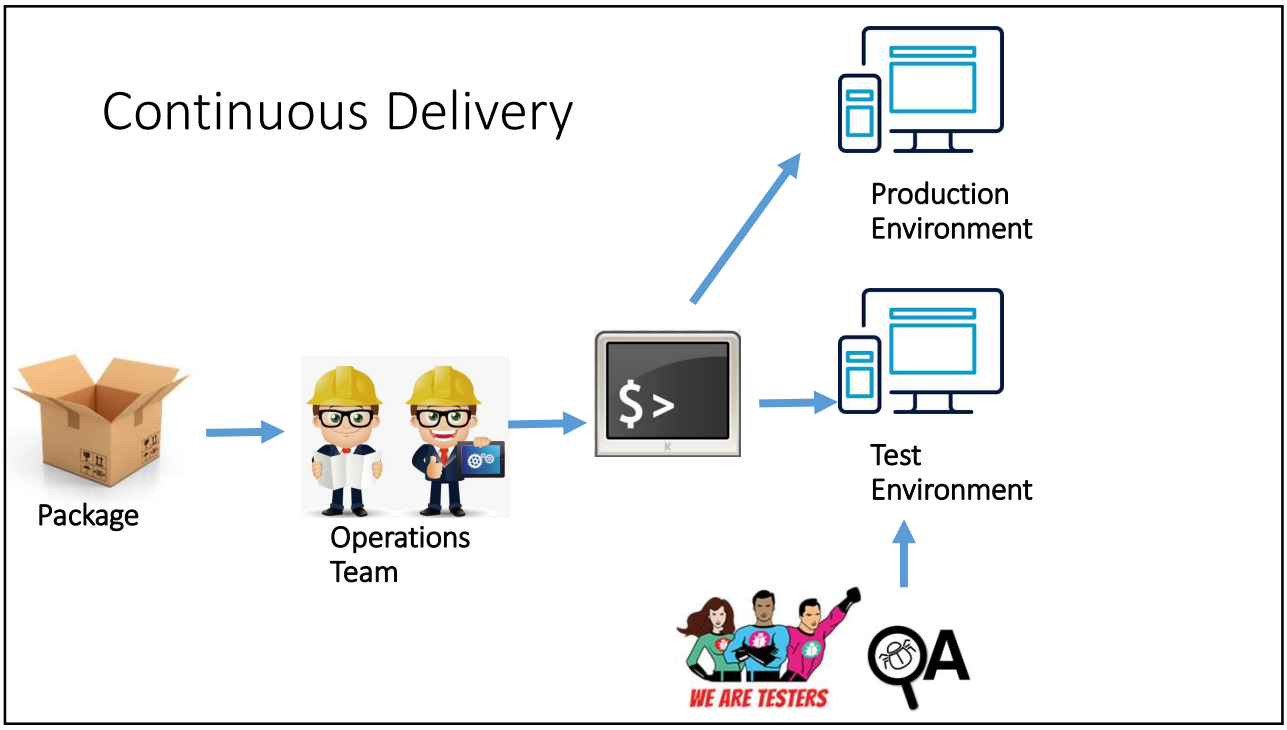
Continuous Delivery

- **Continuous delivery** (**CD**) is a software engineering approach in which teams produce software in short cycles, ensuring that the software can be reliably released at any time and, when releasing the software, doing so manually. It aims at building, testing, and releasing software with greater speed and frequency. The approach helps reduce the cost, time, and risk of delivering changes by allowing for more incremental updates to applications in production. A straightforward and repeatable deployment process is important for continuous delivery.

30

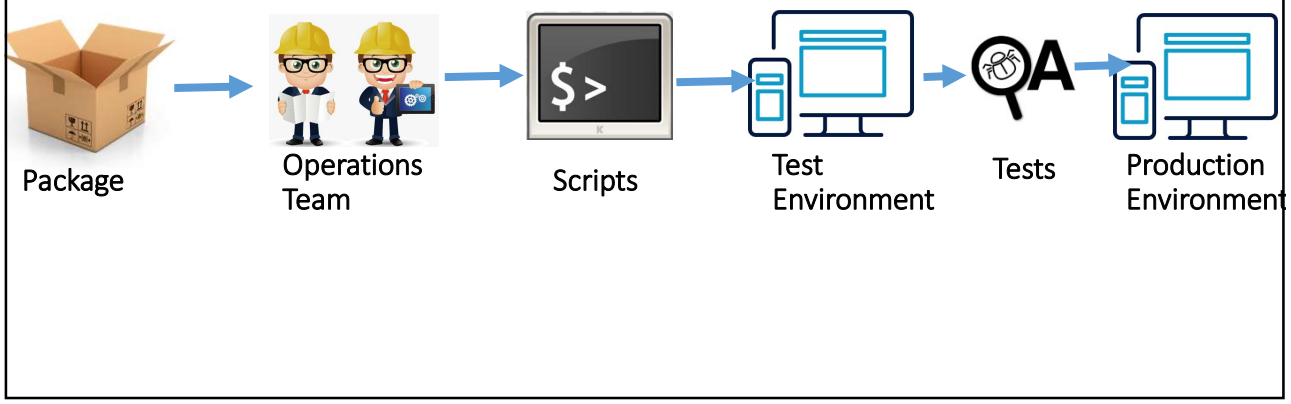

Continuous Delivery
Production Environment
Test Environment
Package
Instructions
Operations Team
WE ARE TESTERS
QA

31


Continuous Delivery
Production Environment
Test Environment
Package
Operations Team
$ >
WE ARE TESTERS
QA

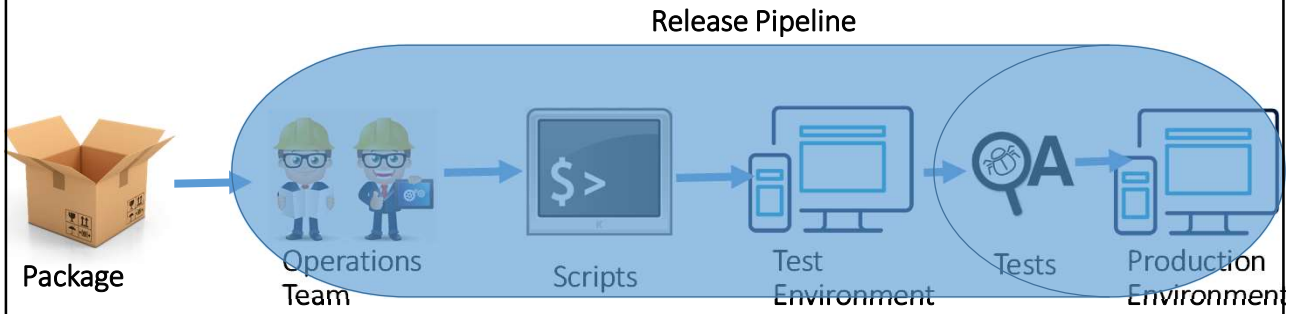
32

Continuous Delivery



33

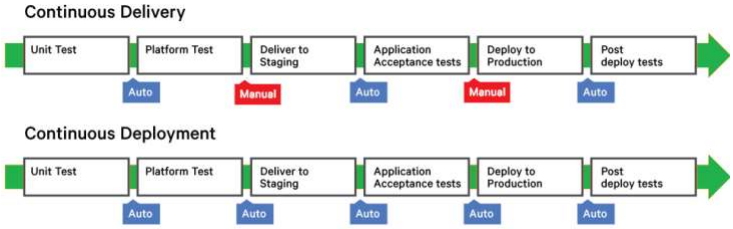
Continuous Delivery



34

Continuous Deployment

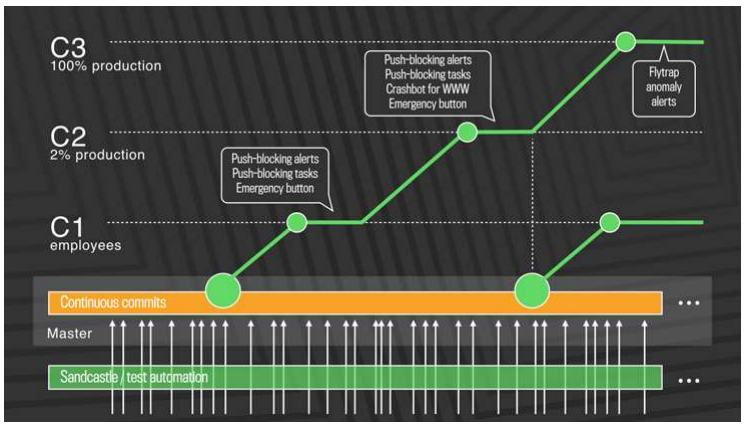
- CD contrasts with continuous deployment, a similar approach in which software is also produced in short cycles but through automated deployments rather than manual ones.



35

Facebook case

https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/.



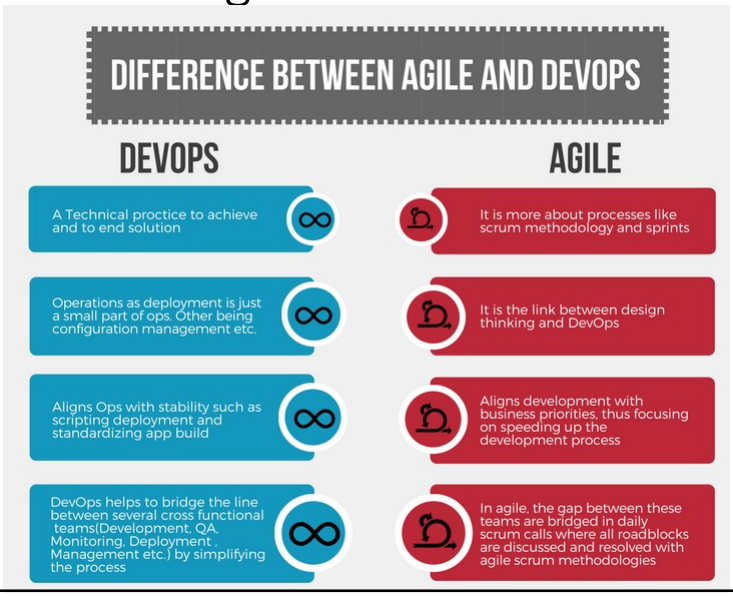
36

DevOps

- Combination of term **Dev**elopment and **Op**eration**s**
- Operation and Integration team are members of Development Team.
- Operation Scripts are at the same repositor than the code (branch for dev, and a branch for Ops).
- Other related terms and cultures like methodology Scrum …

37

DevOps versus Agile



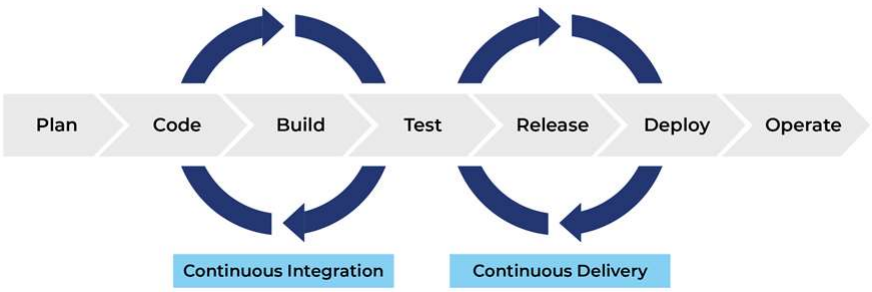
38

DevOps main characteristics

•Configuration Management.
•Continuous Integration.
•Automated Testing.
•Infrastructure as Code.
•Continuous Delivery.
•Continuous Deployment.
•Continuous Monitoring.

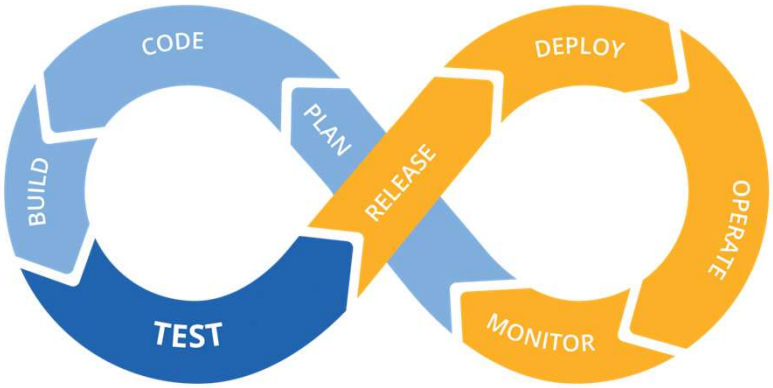
39

Different ways of showing



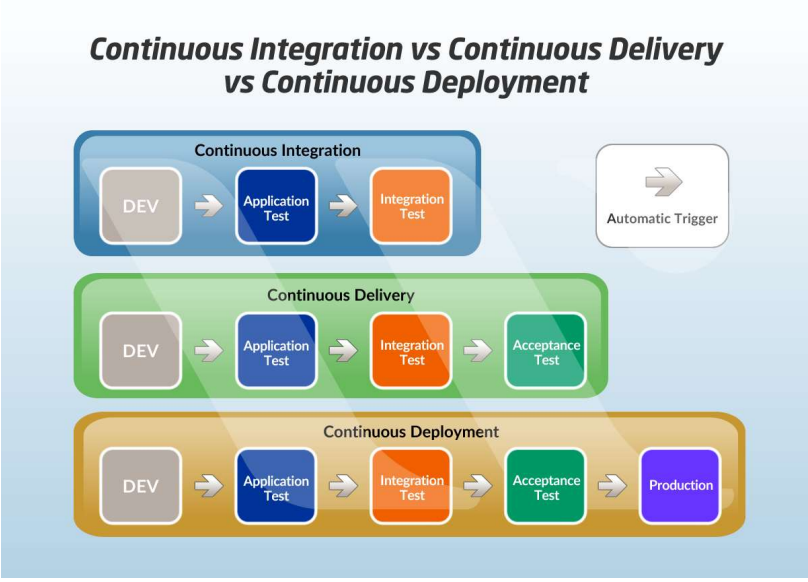
40

Different ways of showing



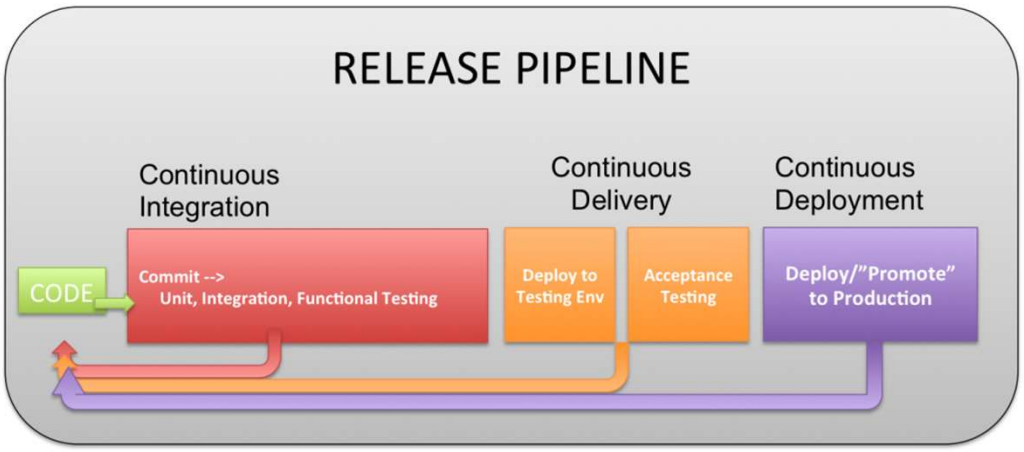
41

DEvOps - Distinct Phase



42

DEvOps - Distinct Phase



43

Tools used in our CI/CD



44

Tools used in our CI/CD





45

Tools used in our CI/CD



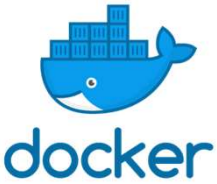




46

Tools used in our CI/CD









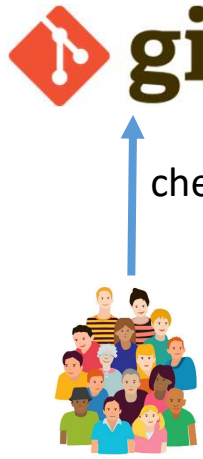
47

CI/CD pipeline flow



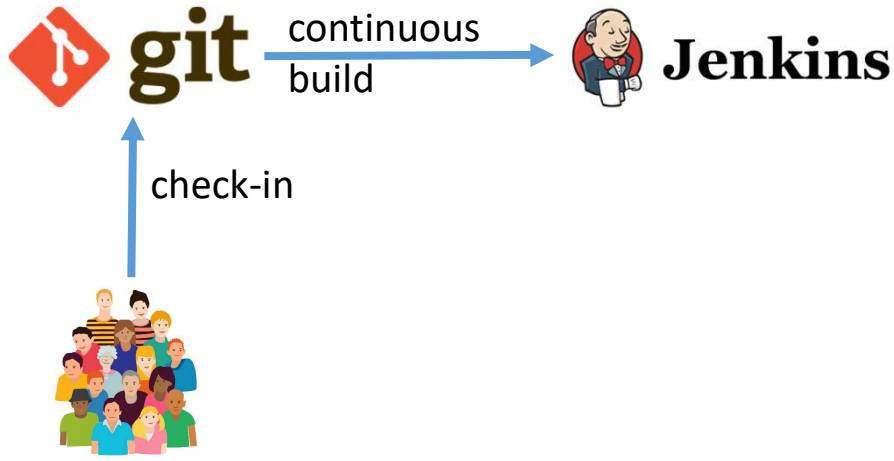
48

CI/CD pipeline flow



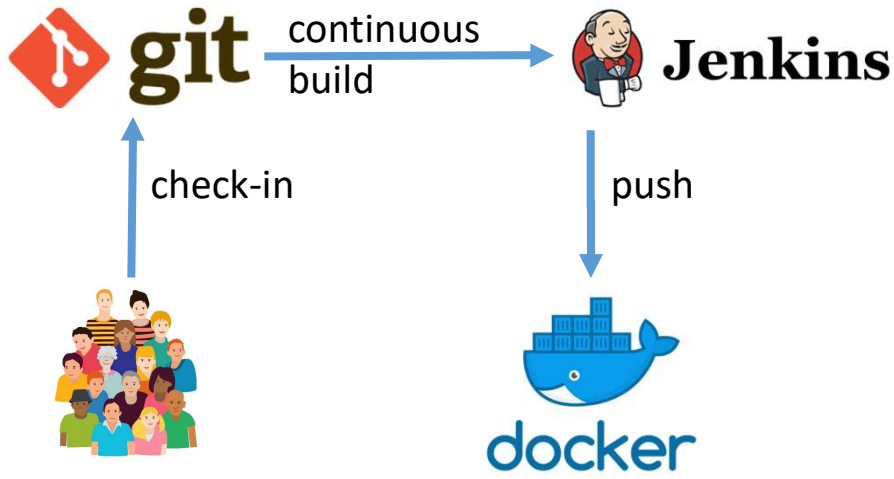
49

CI/CD pipeline flow



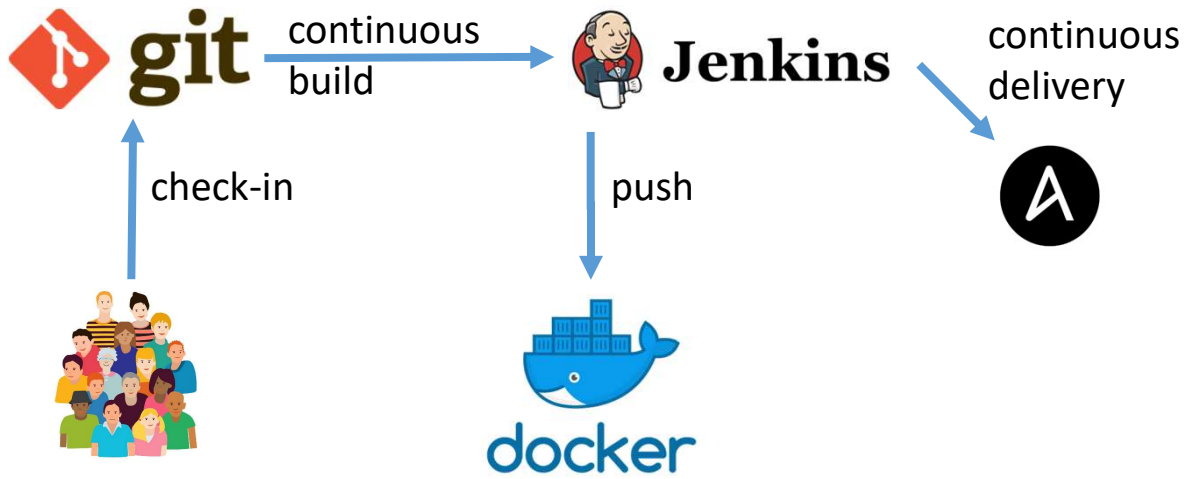
50

CI/CD pipeline flow



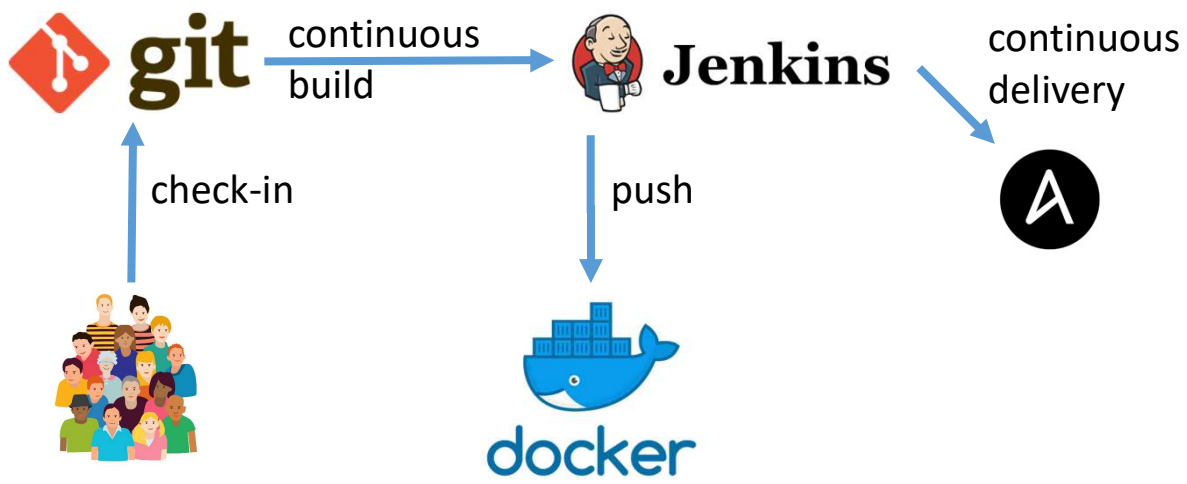
51

CI/CD pipeline flow



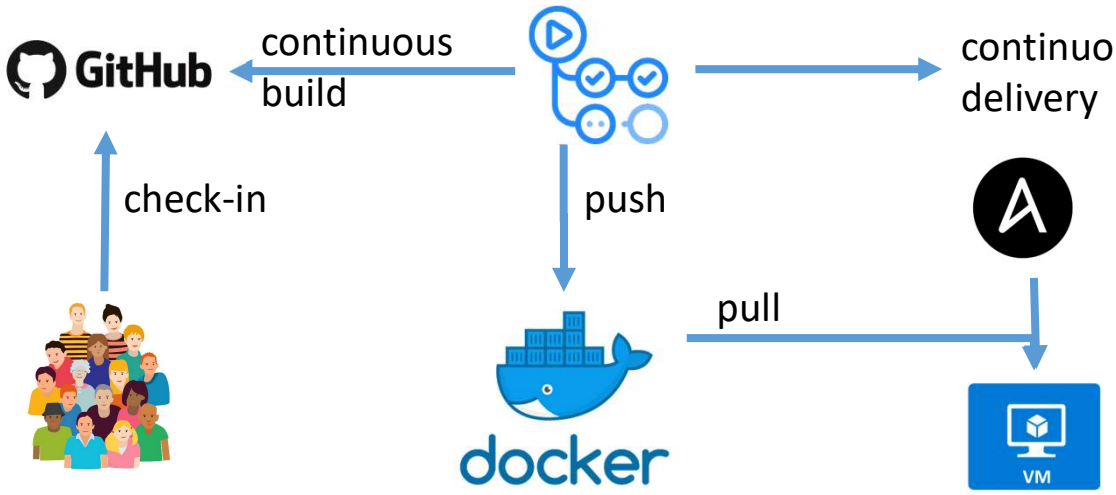
52

CI/CD pipeline flow



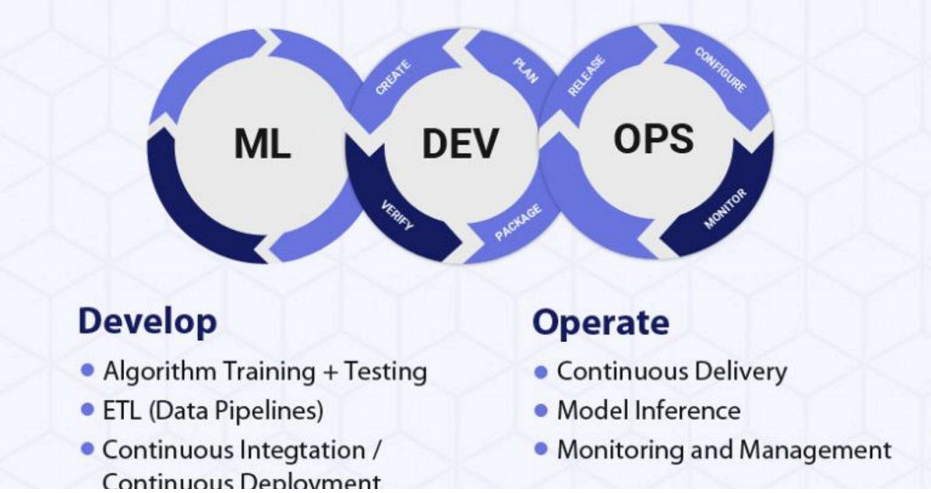
53

CI/CD pipeline flow



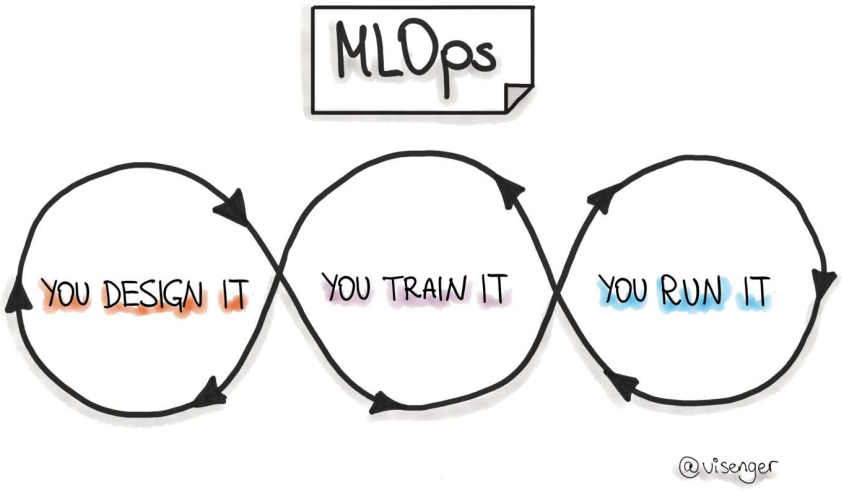
54

MLOps *= DevOps + new incoming data + model retraining*



55

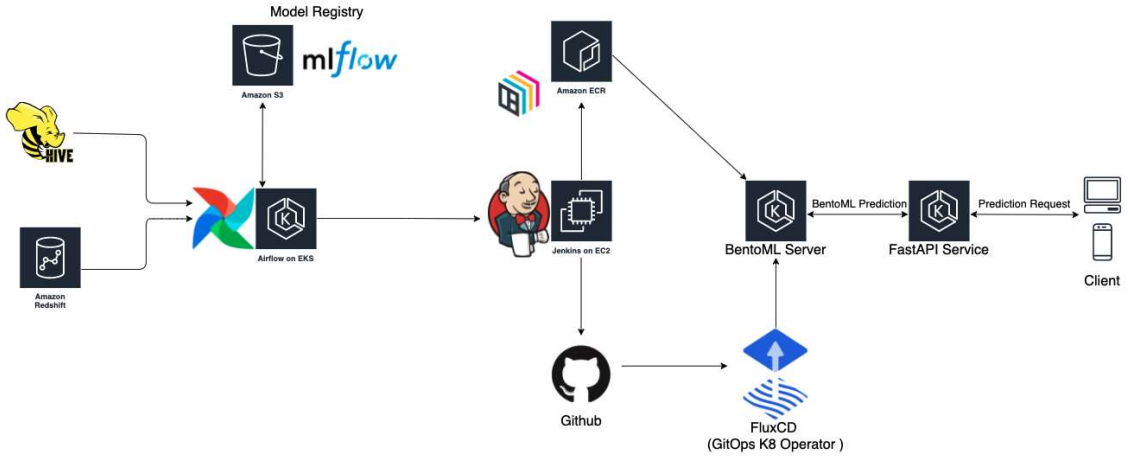
MLOps



https://medium.com/shopback-tech-blog/part-3-model-serving-with-bentoml-and-continuous-retraining-7551776c53f6

56

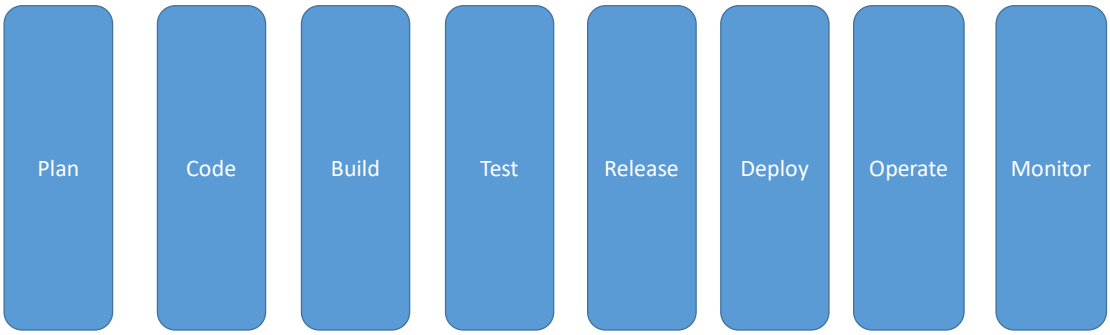
MLOps – architecture example



https://medium.com/shopback-tech-blog/part-3-model-serving-with-bentoml-and-continuous-retraining-7551776c53f6

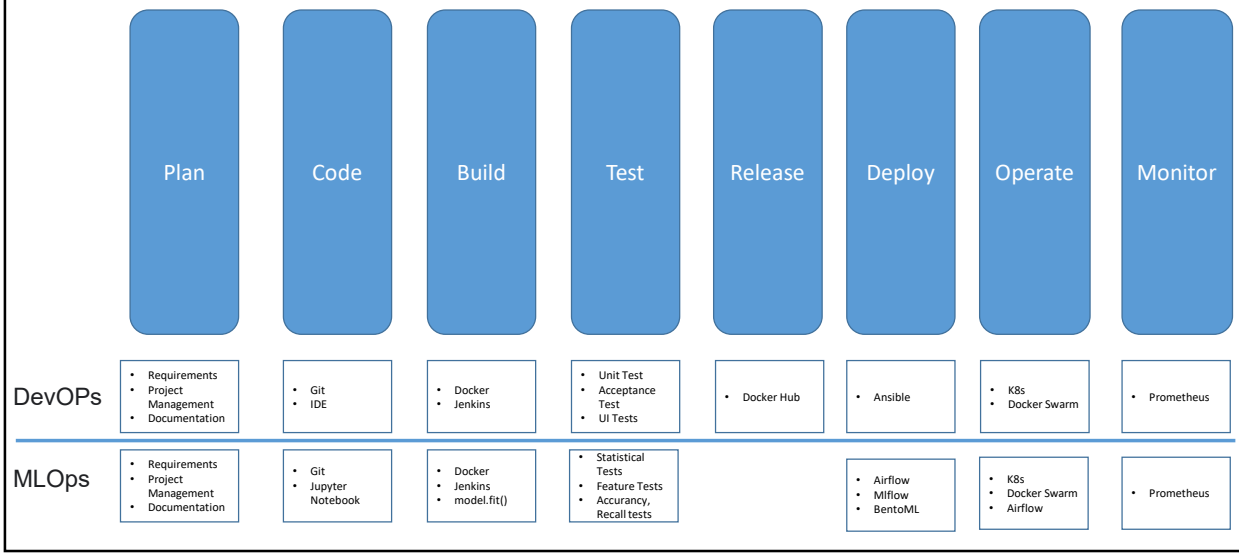
57

MLOps versus DevOps



58

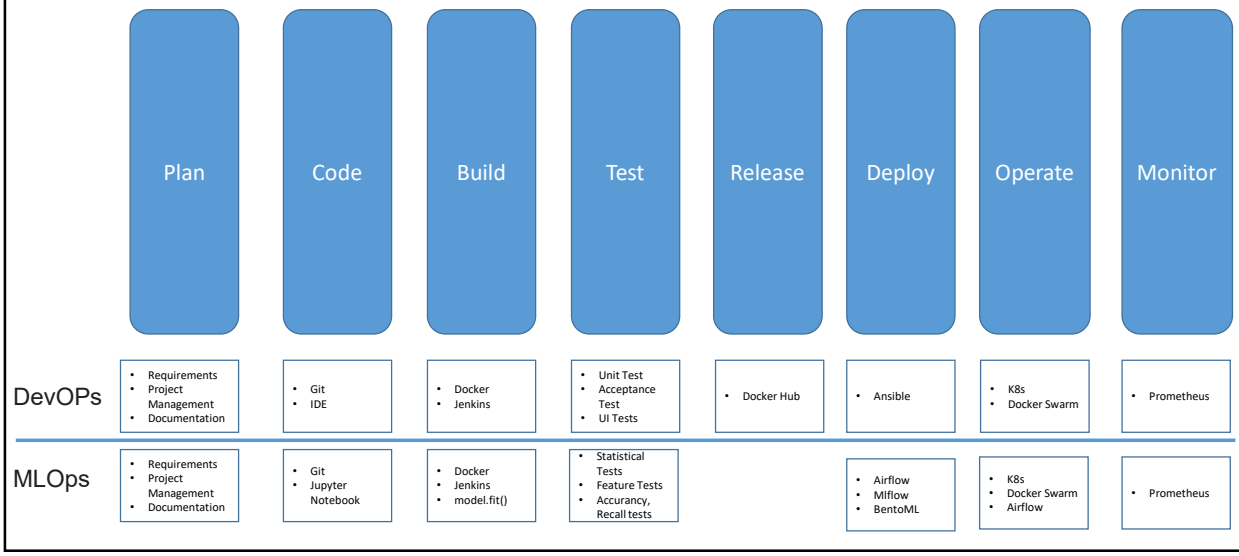
MLOps versus DevOps



59

MLOps versus DevOps

Several key components equal to DevOps and MLOps



60

Some bibliography

- Docker in Practice, 2nd Ed., Ian Miell, Aidan Hbson Sayers, Manning 2019
- Wikipedia
- https://engineering.fb.com/2017/08/31/web/rapid-release-at-massive-scale/

- https://digital.ai/periodic-table-of-devops-tools
- https://www.devops-research.com/research.html#capabilities

- Introducing MLOps: How to Scale Machine Learning, by Mark Treveil, Nicolas Omont, Clément Stenac, Kenji Lefevre, Du Phan, Joachim Zentici, Adrien Lavoillotte, Makoto Miyazaki, Lynn Heidmann

61